Emotet Research **Nov. 10, 2025**

Early-stage research notes — the project evolved beyond this initial scope.

These are follow up notes: Attack Lifecycle, Traffic Analysis & Evasion, some areas may be redundant/repetitive information. This document also includes a Lit. Review, paper listed below

What is Emotet?: a modular malware that enters through phishing, establishes persistence, communicates with C2 servers, steals data, and delivers additional malware as part of a multi-stage attack. It's not detected by what it is, but by how it behaves across system and network data over time. The goal of my model is to learn these patterns — malware doesn't say it's malware, it only leaves patterns in data.

Key characteristics: polymorphic (its code changes slightly each time it's accessed), spreads through spam/phishing emails, can propagate to other devices via brute force, primarily targets the financial sector, and acts as a loader for other malware. Emotet is the gateway — its only job is to install other malware.

Attack Lifecycle:

Stage 1 — Initial Infection (Email & Endpoint Data): Email headers, attachment hashes, macro execution events, process creation logs (winword.exe → powershell.exe → cmd.exe → emotet.exe). Features: parent—child process relationships, suspicious process chains, Office apps launching scripting tools, execution timing. Features to extract for my ML model:

- ➔ Parent—child process relationships
- ➔ Suspicious process chains
- ➔ Office apps launching scripting tools
- ➔ Execution timing (immediate vs delayed)

Stage 2 — Execution & Persistence (Host-Based Data): Registry modification logs, file system changes, scheduled task creation, Windows Event Logs, files written to unusual folders. These are strong signals — normal apps rarely do these quietly. Key features: actions without user interaction, registry write frequency, file path entropy (random-looking names), and persistence mechanism type (registry vs scheduled task).

- ➔ If something is happening without user interaction, it's suspicious. Emotet adds startup entries without you knowing.
- ➔ Registry write frequency. Emotet writes registry entries every time it runs to make sure it survives. High frequency is suspicious.
- ➔ File path entropy (random-looking name). These random names avoid detection and change every infection. A lot of randomness is suspicious.
- ➔ Persistence mechanism type (task vs registry). How malware makes itself run again after restart. Normally, apps will ask permission and use installers. But installers will do this silently and often use multiple methods to achieve the:
  - ➔ Registry, 'run this program on startup'
  - ➔ Scheduled task. 'Run this every X mins'

Stage 3 — C2 Communication (Network Data): Outbound connections, suspicious IP/domain traffic, beaconing patterns. This is where network-based ML peaks. Features: connection frequency per minute, time interval regularity, destination IP reputation, packet size patterns.

Stage 4 — Data Theft & Reconnaissance: Credential access logs, memory artifacts, unusual API calls, email scraping behaviour. Mixed host and application-level data.

Stage 5 — Malware Delivery: Secondary payload downloads, new process trees, new C2 traffic, privilege escalation. Key features: file hash novelty (unseen hashes = new malware), download-to-execution time gap (seconds = suspicious, humans pause but malware doesn't), explosion in process count (150 → 230 = spawning helpers), and sudden traffic volume changes. This happens silently. Typical features:

- File hash novelty — 'how new or unseen is this file'. A hash is a fingerprint, known software has a known hash. But new malware means this hash has never been seen before.
- Download → execution time gap. Think of when you normally download a file. What's not normal is for a file to download and it runs immediately. Humans pause, but malware doesn't.
- Explosion in process count. This means a sudden increase of running programs, if it suddenly jumps from running 150 processes to 230?? This happens due to malware spawning helpers, or dropping payloads.
- Changes in traffic volume. How much data is being sent/received. Malware causes constant background traffic, or sudden large uploads/downloads. This is a strong signal due to unexpected traffic.

Analyzing Logs: Emotet shows up in logs through patterns, not through anything explicitly labelled as malware. For example, winword.exe launching powershell.exe is abnormal — Word shouldn't launch that. Repeated connections to the same IP with similar packet sizes indicate beaconing. The process chain Word → PowerShell → rundll32.exe is one of Emotet's strongest signatures.

- Beaconing indicators: Random IPs (not Google or Microsoft), regular intervals (e.g., every five minutes — humans don't browse like this), and similar packet sizes (an automation pattern).
- Secondary payload indicators: Random file name, download-to-execution gap of only 2 seconds, executed without user action.

Secondary payload download.

- Spotted a random file name
- The download to execution gap is so small. Only 2 seconds.
- It executed without user action.

```
makefile

Time: 10:31:02
Process: powershell.exe
Action: DownloadFile
URL: http://91.214.xxx.xxx/5sd3.exe
SavedAs: C:\Users\j.doe\AppData\Roaming\3kd92.exe

Time: 10:31:04
Process: 3kd92.exe
Action: Execute
```

```
makefile

Time: 10:15:12
SrcIP: 10.0.5.23
DstIP: 185.203.116.45
DstPort: 443
BytesSent: 984

Time: 10:20:12
SrcIP: 10.0.5.23
DstIP: 185.203.116.45
DstPort: 443
BytesSent: 978

Time: 10:25:12
SrcIP: 10.0.5.23
DstIP: 185.203.116.45
DstPort: 443
BytesSent: 982
```

rundll32.exe – one of Emotet's strongest signatures. This is commonly used by Emotet to load malware.

- Word → PowerShell → rundll32

Abnormal Patterns:

- Random IPs. This isn't Google nor Microsoft.
- Every five minutes, humans don't browse at this pattern.
- Similar packet size, this is an automation pattern.

This is beaconing… Emotet is beaconing to a command-control server. This is what checking in repeatedly looks like.

PCAP Analysis – PCAP from malware-traffic-analysis.net:

I downloaded the PCAP zip and analysed the traffic in Wireshark. The first thing I checked for was the repeating IP address. I went to Statistics → Conversations → IPv4.

I found the IP of the internal machine (the victim), where the host talked to many external IPs over time. These two addresses exchanged data over 3 minutes:

- **Internal machine address**: 10.6.13.133 – this is the victim.
- **External server**: 142.250.115.106

I concluded the address of the infected machine from the characteristics of an Emotet attack (first stage to many other attacks):

- The address appears in almost every row.
- It's talking to many external IPs, and it's dominating the traffic I'm seeing.
- But this alone is not enough to prove that it's malware – this is just one of the many characteristics of Emotet. There's a mix of both normal and abnormal traffic, and I need to spot more beaconing. Beaconing is found easily by filtering.

I tried to find beaconing by sorting the duration of this PCAP, but there was none to be found. This file doesn't contain clean C2 beaconing the way I'm familiar with it. There is no periodic C2-style beaconing here – it just looks like normal, safe application and cloud service communication based on this analysis.

I applied the filter: ip.addr == 10.6.13.133 && dns, to spot weird or random domains that might confirm whether this is malware or not. The traffic appeared very normal and safe – it had some noise, but none that was significant or a threat. Details of the traffic I observed:

- wpad.massfriction.com: Normal – used to repeatedly ask if there's a proxy that should be used. It's just noise.
- msftconnecttest.com: Windows uses this to check internet connectivity. It's strange NOT to see this.

- More expected traffic for basic MS services, Windows updates, and logins: login.live.com, windowsupdate.com, bing.com.
- A bunch of CDNs and cloud infrastructure.

If this file were malware with Emotet-style DNS activity, there would be certain patterns to look for:

- Random-looking domains
- Domains queried only a single time
- A low TTL

Conclusion: This PCAP helped me get familiar with analysing traffic using Wireshark, although I'm already familiar with these features and filters. It showed me normal traffic and PCAP behaviour. I couldn't spot any Emotet C2 behaviour, but this could be useful when I train my ML model on normal, clean, and safe PCAP — it works as a good comparison and will come in handy when I begin feature validation. However, for now, it's of no direct use for me.

PCAP Analysis — [CTU-13 Dataset (Stratosphere Laboratory)]

This traffic exhibits Emotet-like C2 behaviour. It's a dataset that is agreed upon and trusted by analysts and many studies related to Emotet. It showcases the behavioural characteristics and nature of an Emotet attack.

- I'm able to acknowledge that this traffic dates back, but it still models Emotet-like C2 behaviour.
- The issue is that these files are from 2011–15, and the concern here is learning the C2 properties and patterns in order to identify the way Emotet behaves in the datasets.
- We can't really say this data is necessarily outdated in this case because we're only concerned with patterns, and patterns don't change over the years. The data will change, but the patterns that are to be identified will remain the same because they represent the style of attack. These properties include: periodic beaconing, low volume and persistent traffic, and encrypted payloads.

I selected Scenario 3, and the key takeaway from this scenario is: what does Emotet-like command-and-control behaviour look like at the network level, and how can it be characterised?

- I downloaded the PCAP file and followed the instructions in the README.
- Infected host: 147.32.84.165 — this is the botnet-infected machine that's going to be exploited.

The normal hosts/IPs (not concerning):

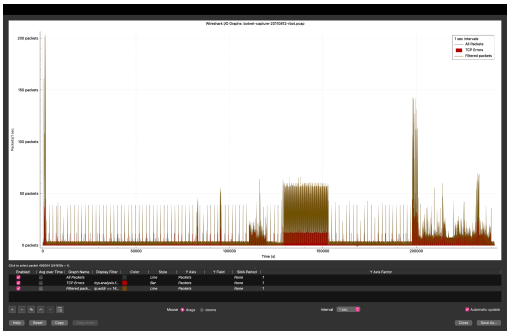
- 147.32.84.170 (bidirectional flows: 217,614 — Label: Normal-V42-Stribrek)
- 147.32.84.134 (bidirectional flows: 1,934 — Label: Normal-V42-Jist)
- 147.32.84.164 (bidirectional flows: 9,160 — Label: Normal-V42-Grill)

- 147.32.87.36 (bidirectional flows: 3,686 — Label: CVUT-WebServer. Not fully reliable as a baseline since it’s a web server.)
- 147.32.80.9 (bidirectional flows: 28 — Label: CVUT-DNS-Server. Not fully reliable since it’s a DNS server.)
- 147.32.87.11

Spotting an interaction like 147.32.84.165 ↔ 95.211.xxx.xxx tells me that the infected internal host is talking to an external system, and this is where C2 usually lives.

I/O Graph Analysis: I analysed the I/O graphs — the spikes represented beaconing. This traffic was used to model Emotet-like command-and-control behaviour. These graphs reveal a lot about the behaviour of this traffic:

- Thin spikes: These represent the low-volume and repeated communication that is a key characteristic of Emotet traffic. In some areas, there’s sustained activity, as well as bursts of activity. All of the above represents the persistent C2 check-in behaviour described earlier. The infected host is “phoning home,” asking for instructions or an order on what to do next, and it’s also regularly sending status info. We can see that it’s trying to maintain a presence and keep the botnet alive — which is Emotet behaviour.
- Sudden tall spikes: C2 is sending new instructions, the client acknowledges and responds, and packets are being exchanged. Since Emotet is a loader, these could also represent the fetch of additional modules, updated configs, or just executing some instruction it’s been given. This causes a momentary surge in traffic, and then it goes back to its quiet beaconing nature. These spikes are NOT how humans browse — it goes quiet, it spikes, and then it goes quiet again. The botnet here could also be attempting to switch servers due to some sort of failure.
- Intermittent high-intensity bursts: The presence of these bursts suggests episodic tasking or payload retrieval, consistent with the modular, loader-based design of Emotet-like malware. This is passive beaconing that later becomes active command execution (the big block).
- Large, consistent block of spikes: A sustained period of high traffic, but it does drop back to the same beaconing that was occurring before. This block is a successful payload download — it’s the transition from an idle bot to an active one, preparing for lateral movement. This is where we see credential harvesting in order to follow the second stage of the malware. It will return back to normal in order to remain present and alive, just normal behaviour — it’s alive and waiting for instruction. Once instructed with something, it’ll do it, and then it will go back to being quiet and hiding.



Document Dropper Architecture – [Emotet, Software S0367 | MITRE ATT&CK®]
Paper: Exploring Emotet's Activities

The document dropper includes five URLs. They're embedded in case one link fails – there's a bunch of others that can still make the infection attempt a success.

1. URLs that host Emotet executables: These lead to the actual program. The sites are usually normal-looking, legitimate websites that have been hacked, or are temporarily used to store the malware file.
2. URLs that host Emotet documents: These documents don't look suspicious. They're used to carry out a successful social engineering attempt in order to deliver the executable. It's the initial malicious document, and it includes normal words that would get the average person to click: PAYROLL, DOC, Smallbusiness, Commercial.

What makes these two different? Each one targets a different victim. The document URLs target the humans – they click them. The executable URLs target the machine – the malware only sees these, and they look random.
Good way to summarise the delivery chain:

1. Email – phishing, get the victim to just open the email.
2. The document URL – looks business-like, innocent, targets the human.
3. Executable URLs – target the machine, random-looking.
4. Finally, the Emotet executable runs.

Code Obfuscation: Emotet breaks its own code on purpose and secretly stores the missing pieces somewhere else in memory, then jumps out to fetch them while running.

- Not the same as packing: Obfuscation happens after loading; packing happens before. Emotet uses both.
- What obfuscation does: Makes the program look broken or incomplete when analysed on disk, but it runs perfectly fine in memory. At runtime, Emotet is fixing itself quietly, but antivirus, disassemblers, and static scanners see nonsense or missing code.
- Why it's done: It breaks static analysis, confuses disassemblers, and makes the job of an analyst much harder – they'd have to analyse the program in a more complex and dynamic manner.

Obfuscation makes Emotet almost impossible to understand while it's on disk. It only makes sense while running. Emotet can only be detected by behaviour – the binaries look different every time. Simply analysing the code or the binaries won't help. If analysed statically, you'll see a broken program that somehow executes correctly.
Recall: Emotet is just the loader. This is why it's the first point of analysis in this project – it's the first entry point to a lot worse. Its only job is to install other malware. It's just the gateway.

How Emotet Modifies Function Structure

A normal program consists of a call for a function, the instructions inside it, and a return. But Emotet changes certain things in this structure:

- Removes the first few instructions of a function, making it look incomplete. This can mean changing the function prologue or setup instructions.
- Replaces normal instructions with a jump to somewhere else (JMP). When the function is called, execution immediately jumps away. If you looked at this program for analysis, you'd naturally assume the function is broken.
- The missing instructions get hidden in a hidden memory area allocated by the loader, completely outside the main executable modules. This area consists of the missing instructions, junk code, and another jump back. This is called the Jump Table.

Emotet at runtime:

1. Function is called.
2. Immediate jump away — instead of a normal running function, we see a JMP to the jump table.
3. Jump table runs the missing instructions, executing exactly what the function should have done.
4. The program jumps back to the original function after finishing the missing setup.
5. Function continues normally — nothing looks odd, and it runs as expected.

Dumping the binaries will fail:

- Dumping the modules results in a broken executable.
- The jump table is not inside the binary — it only exists after the loader runs. Static tools never see the full program.
- From an analyst's perspective, it looks like missing instructions, weird jumps, and broken control flow.

Configuration & C2 Infrastructure: Emotet carries a built-in list of servers it should talk to, and the keys needed to encrypt those conversations. This means Emotet already knows where to call home (IP) and how to connect (through which port).

- It doesn't need DNS to start communicating, and it doesn't need to discover its servers. As soon as Emotet runs, it can start beaconing.
- This is advantageous for Emotet because it can still directly talk to its C2 even if DNS is blocked or monitored.
- Multiple C2s are stored for the same reason that multiple URLs exist in the document droppers — these are fallback servers. If a C2 is down, it'll try its other options, keeping it alive.

RSA Keys: they're embedded in the configuration, and only the attacker has the private key. The public key encrypts the data, and the attacker's private key decrypts it.

- Defenders can see the traffic but can't read it. The stolen data stays confidential and commands can't be forged by the defender.

- This ties back to the fact that the content of the packets does not matter. With Emotet, it’s more important to focus on the patterns of the traffic and its behaviour.

Summary: Emotet embeds its command servers and encryption keys directly in its binary, and communication is done using encrypted web traffic. Traffic behaviour looks normal, and it dynamically loads modules (tools) to steal data and send spam. It’s a loader for exploitative malware.

Dual Infrastructure: Emotet consists of two parallel infrastructures running at the same time. Each includes its own C2 servers, its own RSA encryption keys, and its own deployment schedule. They all drop the same malware, target the same victims, but sometimes operate on alternating days.

- These two infrastructures exist in case one goes down — the other survives and keeps going. It’s also harder to track and take down.
- From the outside, it looks like Emotet is one single thing, but it’s actually two separate things.
- In terms of traffic, Emotet may look similar (timing patterns, burst behaviour, client-server rhythm), but it’s actually coming from different IP clusters with different crypto material.

Different IP Clusters: Emotet has many servers spread around the world, and they change frequently. It can sometimes have two infrastructures at once.

- One day it’ll talk to a certain IP, another the next week, and then a completely different IP after.
- So if your detection is IP-reputation-based, it’s simply not going to work. The IP changes, but the behaviour remains the same.

Different Crypto Material: Emotet embeds different RSA public keys per infrastructure. This means the encrypted traffic from botnet A is cryptographically different from botnet B.

- If an analyst is examining, the packet contents will look completely different, decryption is impossible, and the signatures don’t match.
- But the patterns and behaviours are the same: packet sizes are similar, timing is similar. The lock is different, but the movement is the same.

Implications for ML Detection

Detection cannot rely on IP reputation or packet content. The model needs to detect the pattern, because the pattern stays the same even when the data changes. Key features:

- Timing
- Flow statistics and packet size distributions
- Burst detection
- Sequence modelling
- Connection frequency

The model needs to detect the pattern, because the pattern is going to remain the same even if the data is different.
Emotet's activity is identified through consistent behavioural artifacts across multiple data sources. Analysts will observe:

- Repeated outbound connections to raw IP addresses, rather than domain names.
- Persistent command-and-control communication, and the use of uncommon or shifting ports.
- Such traffic displays stable patterns in timing, volume, and request structure.

Refresher Video (break): What is Emotet?

- Used to steal credentials, sensitive information, and financial data.
- Can also load other malware onto the victim's device to spy on you
- Polymorphic nature: every time it's accessed, its code changes just a bit. This makes it easy to hide.
- Once a victim is infected, it can spread to other devices in the network, and it also uses brute force to crack passwords.
- Mainly spreads through spam, phishing email.
- Used to target the financial sector.

These are some definitions I noted down while doing my research for the Emotet section. Certain terms were recurring in many papers, sources, data, packets, etc.. I kept track of them here in case I needed any refresher while working, researching, and note taking.

- I do NOT take credit for the wording of all these definitions, I have used google and AI to take note of some of these definitions.

Definitions:

- **Remote attack servers**: computers on the internet that hackers control. When Emotet makes it to your machine, it needs instructions; what should I steal? Should I download more malware? Do I wait? Emotet will contact the attacker's server so it has some sort of instruction on what to do. Like asking your boss what to do next, a remote attack server is the boss.
- **PCAPs**: recordings of internet traffic. Packet Capture, each 'packet' is a small piece of data sent over the network. PCAPs are needed for Emotet because if Emotet talks to attacker servers, that convo travels over the network. PCAPs capture that convo, and the PCAPs are used to see things like: Which server Emotet contacted, when it contacted it, and how often it checks in.
- **Outbound network connections**: when your computer sends data OUT to the internet. Inbound is data coming to you, and outbound is data leaving from you. With Emotet, outbound connections are made silently in the background. Example with Emotet: Your laptop sends data to: 185.xxx.xxx.xxx (attacker server), every few minutes… but you aren't doing anything. That's sus.
- **Secondary payload data**: extra malware that gets downloaded after the first infection. This is often <u>ransomware, password stealers, and network-spreading malware</u>. Recall how Emotet works:
    1. Emotet infects the computer
    2. It will connect to attacker servers
    3. Attacker says 'download this new malware'
    4. Emotet downloads and runs it. That new malware = secondary payload.
- **Registry**: like a control panel database. Most normal apps will read to it, they rarely write to it. This is significant because malware writes to it a lot in order to start on reboot and stay hidden. This is what makes the registry write frequency high, which is suspicious.
- **File path entropy:**
    - ➔ Normal software: C:\Program Files\Google\Chrome\
    - ➔ Malware: C:\Users\AppData\Roaming\xJ92kLq0\A8f9.exe
- **Beaconing**: when the infected computer contacts the same server again and again. Emotet does this to ask if there's any new instructions. This may be contacting every 5 minutes, if it's the same IP, same packet size.
- **Destination IP reputation**: if you speak to a suspicious server, like a random IP in another country, that's sketchy and suspicious. There are several known bad servers.
- New process trees: new chain of programs launching other programs. Office apps SHOULDN'T launch system tools.
- **Normal**: Chrome → website
- **Malware**: Word → Powershell → CMD → unknown.exe
- **Internal hosts** = victims, peers, infrastructure
- **External IPs** = potential C2 servers
- **C2**: command and control, the system attackers use to remotely control infected machines.